

Scalability & Future Plan

j

Date	29 June 2026
Team ID	
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	1 Mark

Scalability & Future Plan:

The Rising Waters Flood Prediction System is designed as a machine learning-based web application that predicts flood occurrence using historical weather and rainfall data. While the current implementation successfully performs local flood prediction through a Flask application, there are several opportunities to improve scalability, enhance system performance, and expand functionality for real-world deployment. This document outlines the current limitations of the system, proposed scalability improvements, and the future development roadmap.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address (High / Medium / Low)
1	The system uses a historical dataset and does not integrate real-time weather information.	Predictions are limited to manually entered data and cannot respond to live weather conditions.	High
2	The application is deployed only on a local Flask development server.	The system cannot currently support remote users or multiple concurrent users over the internet.	High
3	The dataset is relatively small and contains a limited number of meteorological features.	Model accuracy and generalization may decrease when applied to larger or more diverse geographical regions.	High

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Supports a single user on a local Flask server.	Deploy the application on IBM Cloud, AWS, Azure, or Render with a production-ready WSGI server (e.g., Gunicorn) to support multiple concurrent users.
2	Data Storage	Uses a static historical dataset stored locally.	Store datasets in a relational or cloud database and periodically update them with recent rainfall and weather information.
3	Performance	Prediction is performed locally after loading the saved model and scaler.	Optimize preprocessing, implement caching where appropriate, and upgrade server resources to improve response time under higher workloads.
4	Security	No authentication or encryption is implemented because the application is intended for local demonstration.	Add user authentication, HTTPS, secure input validation, and role-based access control before public deployment.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 1	Deploy the Flask application to a cloud platform (IBM Cloud, AWS, Azure, or Render).	1–2 Months	Make the application accessible online for m
Phase 2	Integrate real-time weather APIs to automatically retrieve rainfall and meteorological data.	2–4 Months	Improve prediction accuracy by using live environmental information.
Phase 3	Develop an interactive dashboard with maps, charts, and historical flood analysis.	4–6 Months	Improve visualization and decision-making for disaster management authorities.
Phase 4	Add automated SMS, email, or mobile notifications for early flood warnings.	6–12 Months	Enable timely alerts to citizens and emergency response teams, improving disaster preparedness.